
Intuitively Teleoperating a Mobile Robot with Eye Gaze Tracking

Jeon, Jihyun

California Institute of Technology

Mentors: Soon-Jo Chung, Yujin An

1 Abstract

The development of brain-computer interface (BCI) has introduced a new possibility in helping those with severe motor disabilities by enabling users to control devices to aid in daily tasks by merely thinking about the task they want performed. However, many of the current BCI research requires the user to make many lower-level commands, such as turning left and right, instead of providing an intuitive way to utilize the robot with higher-level decisions, such as moving to a table. In this project, we aim to overcome this limitation by creating an intuitive way of controlling a robot through a Virtual Reality (VR) interface. The VR interface will use eye tracking data from the user to recognize user intent, which will be combined with the BCI data as inputs to a Sequential Convex Programming (SCP) based motion planner. Future work on this project will include comparing the use of the VR interface with BCI to direct control and to the use of only BCI.

2 Background and Introduction

The development of brain-computer interface (BCI), or direct communication mechanism between the human brain and external devices like computers or robots, has introduced a new possibility in reconstructing the basic ability to perform Activities of Daily Living (ADLs) for those with severe motor disabilities, like paralyzed people or elderly people. BCI systems show promise in enabling users to control devices to aid in daily tasks by merely thinking about the task they want performed. There have been various attempts to develop a BCI system for robots, such as mobile robots used to help users [6, 12], or robotic arms capable of performing basic tasks [13, 2].

However, many of the current BCI research requires the user to make many lower level commands, such as turning left and right, instead of providing an intuitive way to utilize the robot with higher level decisions, such as moving to a table. While BCI aims to gather user intent from brain signals, it is only able to do so for a limited number of discrete commands.

There have been attempts to recognize user intent from eye tracking data, especially to control robots. Many of these applications are for collaborations between robots and humans [1, 5]. However, many of the existing research is for manipulation tasks and is set in an unchanging environment, oftentimes requiring the user to be able to see the environment the robot is set in. This creates a limitation for the use cases as the user is unable to teleoperate robots from distant locations.

Thus, this research project aims to overcome the limitations of the current BCI and eye gaze research by combining them, using eye gaze to recognize user intent while combining it with BCI to improve the reliability of the data. The user intent will be used to control a mobile robot.

This will create an improved, intuitive VR interface for teleoperating robots aimed to be used by people who have trouble doing motor actions. It will function similarly as existing teleoperation research that allows the user gain real time visual feedback from the robot [8, 3] but will only be controlled using eye gaze and EEG data, allowing the user to remain stationary. It will also improve

36 on existing projects of controlling robots using eye gaze [4, 11] by allowing the user to give higher
37 level target commands rather than low level commands.

38 However, there were several challenges that prevented the implementation to be completed within the
39 time frame. The initial approach to implement a native VR interface failed due to several difficulties
40 as illustrated in 3.1. Unity was then chosen to implement the VR interface, but this also came with
41 challenges as shown in 3.2. This leaves several future tasks of finishing the implementation and
42 potentially incorporating the VR with a BCI interface.

43 3 Methods and approaches

44 3.1 Native VR Interface

45 A virtual reality (VR) interface is used to gather real time eye gaze data from the user. The Au-
46 tonomous Robotics Control Lab (ARCL) currently has a Meta Quest Pro that is used for this project.
47 The Quest Pro is capable of face tracking, hand tracking, and most importantly, eye tracking.

48 Initially, the VR program was developed as a native application on Linux so it can be incorporated
49 into the ongoing BCI research at the ARCL lab. As the BCI interface requires minimal latency in
50 receiving user data, the goal was to develop the VR interface on a native platform as opposed to using
51 popular game engines like Unity or Unreal for minimal overhead and optimized execution.

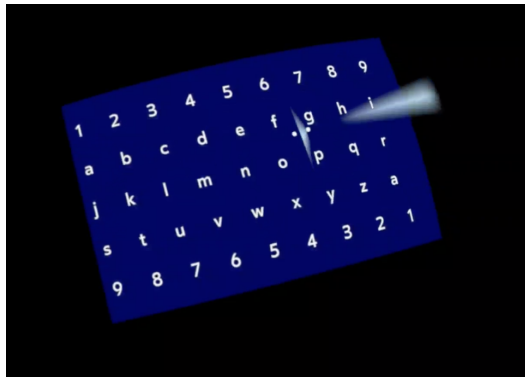


Figure 1: Native eye tracker

52 First, the sample code of the Meta OpenXR plugin was modified to implement a preliminary eye
53 tracker. The eye tracking program shows rays that represents the gaze of each eye of the user and
54 when the ray collides with an object in the scene, it keeps track of the location of the collision (i.e.
55 the location that the user is looking at).

56 However, further development of a native application, especially in integrating it with a Intel Re-
57 alSense D455 depth camera stream, came with many unexpected challenges. Several approaches
58 were taken but ultimately none of them were successful, which motivated the decision to use a Unity
59 based application.

60 3.1.1 Exoplayer

61 The initial attempt was to use Exoplayer. Exoplayer is an implementation of a media player interface
62 provided by Android. The API can be used to render various media types, including streaming videos
63 using Real-Time Streaming Protocol (RTSP) with Real-Time Transport Protocol (RTP) over a User
64 Datagram Protocol (UDP) unicast. As Exoplayer is the API used by many Android applications,
65 including YouTube, for various operations related to playing media in apps, it was the first choice for
66 the project. However, the challenge with Exoplayer was its lack of support for native development,
67 VR integration, and connection to RealSense.

68 3.1.2 ROS Java

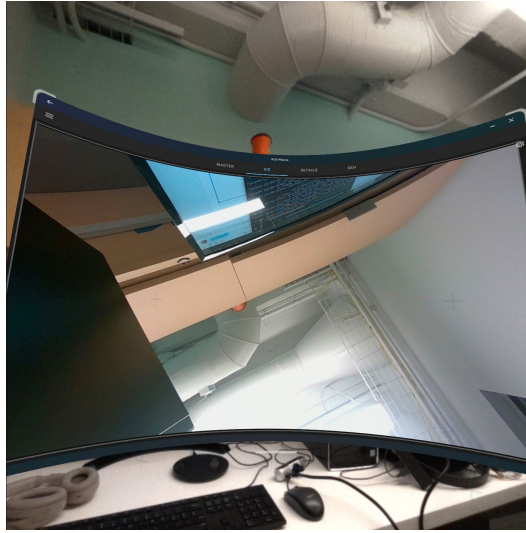


Figure 2: ROS Mobile displaying RealSense camera stream on the VR

69 The next attempt was to use ROS 1. ROS Mobile [7] was first used to test the capabilities of ROS for
70 the purpose of the project. ROS Mobile is an Android application designed to communicate with
71 a robotic system using ROS1 messages. It provides capability to subscribe to a camera node and
72 display the camera messages. As the Quest Pro has the capability of running Android mobile apps,
73 the ROS Mobile app was run on the Quest Pro and used to display the RealSense camera stream as
74 show in 2.

75 The ROS Mobile app is unable to be used directly for this project due to its inability to include VR
76 specific functions like eye tracking, it served as a good resource for how to use ROS1 to display video
77 streams in an Android based app. The pipeline is to use ROS Java to receive camera messages from a
78 RealSense node, convert the camera message into a bitmap, and displaying the bitmap in the Android
79 app. However, ROS Java is an archived, outdated project that is incompatible with any VR app that
80 OpenXR. While there were numerous attempts to overcome this incompatibility, they were ultimately
81 unsuccessful.

82 3.1.3 GStreamer

83 The final attempt was to use GStreamer. GStreamer is a media processing framework that has many
84 capabilities, especially for streaming over networks. It has good support for RTSP streaming and it
85 also provides many resources for using GStreamer for Android NDK development, making it a better
86 candidate than Exoplayer.

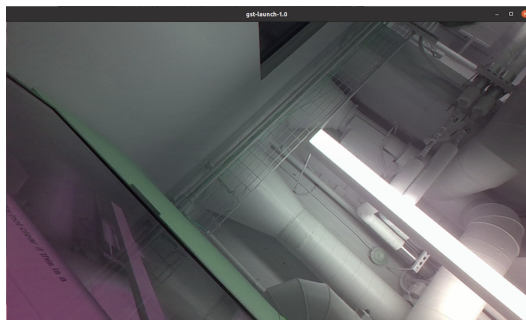


Figure 3: Use of RealSense as a source for GStreamer

87 Before GStreamer was used in the VR app, it was first tested to make sure the RealSense camera feed
88 can be streamed over network. While the RealSense camera was able to be connected to GStreamer
89 3, it failed in being streamed over network using UDP due to the incompatibility of the encoding type
90 of RealSense and the encoding required by GStreamer.

91 3.2 Unity VR Interface

92 Due to the many challenges of native VR app development, it was decided to use Unity to develop
93 the app instead.

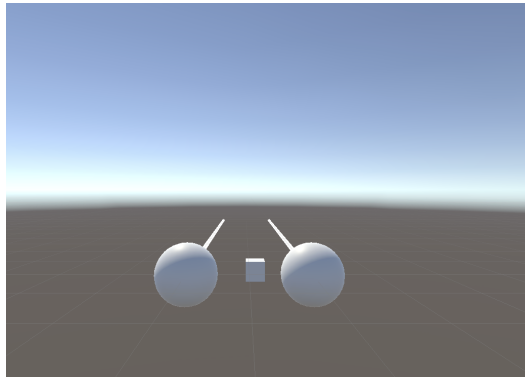


Figure 4: Eye tracker implemented in Unity

94 Using the Meta XR SDK Unity assets, an eye tracker was also developed with Unity, as seen in 4.
95 The spheres represent each eye of the user and each have a ray attached to it which represents
96 the gaze of each eye. The sphere game objects had the OVREyeGaze component attached to them,
97 meaning their transform was updated in real time to correspond to the rotation of the eyes. Then, if
98 and where the rays collided with another object kept track of. This corresponds to where the user is
99 looking in the scene.

100 While the initial plan was to add a real time camera stream to the interface, the plan changed to
101 adding AWSIM [9] for ease of implementation, ease of testing and potential applications to research
102 being done at ARCL using AWSIM. However, using AWSIM created several problems that could not
103 be resolved within the SURF time frame.

104 The initial problem came from the incompatibility between AWSIM and the Oculus Quest Pro.
105 The Quest Pro uses Android as its base operating system. But AWSIM is intended to be used as a
106 computer simulation. Thus, the graphics that AWSIM required were available as an Android app.

107 To avoid this problem, the Quest Pro can be used as a PCVR headset using Oculus Link or Steam
108 Link. This means the AWSIM program can run as a Windows program and this can be streamed to
109 the Quest Pro by connecting the Quest Pro to the computer, either wirelessly using Steam Link or
110 with a cable using Oculus Link.

111 However, this didn't work successfully. The connection, both using Steam Link or Oculus Link,
112 requires a high spec computer and even when using a computer that meets the technical requirements,
113 is very unreliable. It was also difficult to combine the eye tracking components of the Meta SDK,
114 which are meant for an Android based VR app, with the AWSIM PC program. Even after they were
115 combined and able to be compiled, due to an unknown bug, the eye tracking did not seem to work
116 over the PCVR app.

117 4 Conclusion and Future Work

118 As the research was unsuccessful in being completed within the SURF time frame, there are several
119 future tasks and potential work.

4.1 Incorporation with Camera Stream

As the use of AWSIM in VR was ultimately unsuccessful, it would be better to use the camera stream from the RealSense instead of trying to work in simulation. The RealSense has been tested as a streaming source and there are examples of Unity projects that used a VLC player plugin [10] to receive streams. This is a promising approach for combining the eye tracker VR app with the RealSense real time camera feed.

4.2 Motion Control

The eye gaze data can be used for navigational control by using a Proportional Integral (PI) controller. Given a eye gaze point, the PI controller will aim to navigate such that the eye gaze point goes to the center of the screen. Motion control with the VR interface can be tested against controlling with buttons or with a driving wheel to test its feasibility as a control method.

4.3 BCI Research

A further application of the project would be to use the VR interface with a BCI interface. Potentially, the VR interface would provide information on where the user wants a mobile robot to move. This information can be combined with a BCI system so more user intent can be recognized. The BCI system can be used to differentiate between when the user wants the robot to move and when they want the robot to stop. This is an improvement on many of the current BCI robot control systems and holds potential for a full reliable teleoperation system of a mobile robot.

5 Acknowledgments

I'd like to thank Yujin An for her guidance and support throughout the project, Dr Soon-Jo Chung for the opportunity to work in the ARCL lab, and the SURF program for funding my research.

References

- [1] V. Belcamino, M. Takase, M. Kilina, A. Carfi, F. Mastrogiovanni, A. Shimada, and S. Shimizu. Gaze-based intention recognition for human-robot collaboration. In *Proceedings of the 2024 International Conference on Advanced Visual Interfaces, AVI '24*, New York, NY, USA, 2024. Association for Computing Machinery.
- [2] L. Cao, G. Li, Y. Xu, H. Zhang, X. Shu, and D. Zhang. A brain-actuated robotic arm system using non-invasive hybrid brain-computer interface and shared control strategy. *Journal of Neural Engineering*, 18(4), 2021.
- [3] X. Cheng, J. Li, S. Yang, G. Yang, and X. Wang. Open-television: Teleoperation with immersive active visual feedback, 2024.
- [4] D. Gêgo, C. Carreto, and L. Figueiredo. Teleoperation of a mobile robot based on eye-gaze tracking. In *2017 12th Iberian Conference on Information Systems and Technologies (CISTI)*, pages 1–6, 2017.
- [5] C.-M. Huang and B. Mutlu. Anticipatory robot control for efficient human-robot collaboration. In *2016 11th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, pages 83–90, 2016.
- [6] A. C. Lopes, J. Rodrigues, J. Perdigão, G. Pires, and U. J. Nunes. A new hybrid motion planner applied in a brain-actuated robotic wheelchair. *IEEE Robotics & Automation Magazine*, 23(4):82–93, 2016.
- [7] N. Rottmann, N. Studt, F. Ernst, and E. Rueckert. Ros-mobile: An android application for the robot operating system. *arXiv preprint arXiv:2011.02781*, 2020.

- 162 [8] P. Stotko, S. Krumpfen, M. Schwarz, C. Lenz, S. Behnke, R. Klein, and M. Weinmann. A vr
163 system for immersive teleoperation and live exploration with a mobile robot. In *2019 IEEE/RSJ*
164 *International Conference on Intelligent Robots and Systems (IROS)*, pages 3630–3637, 2019.
- 165 [9] tier4. Awsim.
- 166 [10] VideoLan. Vlc for unity.
- 167 [11] G. S. Watson, Y. E. Papelis, and K. C. Hicks. Simulation-based environment for the eye-
168 tracking control of tele-operated mobile robots. In *Proceedings of the Modeling and Simulation*
169 *of Complexity in Intelligent, Adaptive and Autonomous Systems 2016 (MSCIAAS 2016) and*
170 *Space Simulation for Planetary Space Exploration (SPACE 2016)*, MSCIAAS '16, San Diego,
171 CA, USA, 2016. Society for Computer Simulation International.
- 172 [12] B. Xu, D. Liu, M. Xue, M. Miao, C. Hu, and A. Song. Continuous shared control of a
173 mobile robot with brain–computer interface and autonomous navigation for daily assistance.
174 *Computational and Structural Biotechnology Journal*, 22:3–16, 2023.
- 175 [13] R. Zhang, S. Lee, M. Hwang, A. Hirakana, C. Wang, W. Ai, J. J. R. Tan, S. Gupta, Y. Hao,
176 G. Levine, R. Gao, A. Norcia, L. Fei-Fei, and J. Wu. Noir: Neural signal operated intelligent
177 robots for everyday activities. In *7th Conference on Robot Learning*, Atlanta, 2023.